

輪読レポート（2026 年 5 月 12 日）への解説・訂正

Vincent-Lamarre et al. (2016). “The Latent Structure of Dictionaries.” *Topics in Cognitive Science* 8(3): 625 – 659. DOI: 10.1111/tops.12211

はじめに

全体の流れ — Kernel・Core・MinSet・NP 困難性の説明 — はよく読み取れています。ただし実装に直結する技術的誤りがいくつかあります。重要度の高い順に解説します。

1. Kernel の計算方向が逆 △ 最重要

あなたの記述：「*in-degree* が 0 の単語を繰り返し除去する」

原著の正しい記述：out-degree が 0 の単語を繰り返し除去する。

なぜ方向が重要か

辞書グラフの辺の向きは次のように定義されています：

add_edge(u, v) の意味：「u が v の定義文の中に現れる」
(u は v を定義する側)

out-degree が 0 の単語とは、「どの単語の定義にも使われていない単語」つまり定義を消費するだけで、他の単語を定義することに一切貢献しない語です。こういう語を取り除いても、他の語の定義可能性には影響しません。

in-degree が 0 の単語とは、「辞書内のどの定義文にも登場しない単語」（＝他の語から定義されない語）であり、まったく別の概念です。

具体例で確認する

グラフ：

fast → speed → moving → fast (3 つの単語が互いを定義する閉路)
fast → quickly (quickly の定義に fast が使われている)

辺の意味：

fast → speed : 「fast」は「speed」の定義に現れる

speed $\rightarrow$ moving : 「speed」は「moving」の定義に現れる
moving $\rightarrow$ fast : 「moving」は「fast」の定義に現れる
fast $\rightarrow$ quickly : 「fast」は「quickly」の定義に現れる

out-degree : fast=2, speed=1, moving=1, quickly=0
in-degree : fast=1, speed=1, moving=1, quickly=1

正しいアルゴリズム (out-degree = 0 を除去) : 1. *quickly* を除去 (out-degree = 0) 2. 他に out-degree = 0 の語がなければ終了 3. **Kernel** = {fast, speed, moving} ✓

あなたのアルゴリズム (in-degree = 0 を除去) : 1. in-degree = 0 の語が存在しない $\rightarrow$ 何も除去されない 2. “**Kernel**” = {fast, speed, moving, quickly} ✗

quickly は「fast を使って定義される」だけの末端語であり、何かを定義しません。それでもあなたのアルゴリズムでは Kernel に残ってしまいます。

原著のアルゴリズム (Blondin Massé 2008, Algorithm 2)

$U \leftarrow \{\}$

繰り返す :

$U \leftarrow U \cup \{v \in V \mid N^+(v) \subseteq U\}$ # $N^+(v)$ = v の out-隣接点の集合

変化がなければ終了

Kernel $\leftarrow V \setminus U$

$N^+(v) \subseteq U$ は「 v の out-neighbor がすべて除去済み集合 U に含まれる」= 残りのグラフで v の実効 out-degree がゼロ、という意味です。

2. Core の定義 — 最大 SCC とは限らない

あなたの記述 : 「*Kernel* 内の最大 SCC (強連結成分) が *Core* である」

原著の正しい記述 : Core は Kernel の condensation DAG 上で in-degree = 0 の SCC (Sources) の和集合です。

論文が解析した 4 つの辞書のうち 2 辞書では Core = 最大 SCC ですが、残り 2 辞書では「最大 SCC + 少数の小さな SCC」が Core になります。著者たちはこれを前処理の副産物と見なしつつも、定義は Source SCCs としています。

実装では :

```
C = nx.condensation(Kernel_subgraph)
sources = {n for n in C if C.in_degree(n) == 0}
Core = sources の各 SCC に属する単語の和集合
```

3. MinSet の計算 — greedy ではなく ILP + CPLEX

あなたの記述：「大規模辞書には *greedy* (近似) を使う」

原著の正しい記述：全 4 辞書とも **ILP (整数線形計画)** を **CPLEX** で解いています。Webster (約 24.8 万語) は「数日かけてほぼ最適解を得た」と書かれており、*greedy* アルゴリズムは使われていません。

Greedy (最多閉路参加語を繰り返し除去) は妥当な近似ですが、原著の手法ではありません。実装で近似版を作る場合は、`optimal=False` フラグを立てて区別してください。

4. 前処理 — stemming (語幹還元)、lemmatization ではない

あなたの記述：「*Lemmatization* : 各単語を *base form* に変換する」

原著の正しい記述：論文では “stemmatized” (stemming) という用語が使われています。

処理	例	特徴
Stemming	“running” → “run”	辞書に存在しない形も出る
Lemmatization	“running” → “run”	常に正しい語彙形、品詞が必要

本研究の実装では WordNet との語彙整合性を優先して **lemmatization** を採用します。これは原著からの意図的な逸脱であり、論文の方法論で明記します。

また原著の “first sense” は「品詞ごとの最初の語義」です。たとえば “bank (名詞)” と “bank (動詞)” はそれぞれ別エントリとして扱い、それぞれの品詞で最初の語義を使います。

5. 出版年と使用辞書

あなたの記述：「*Vincent-Lamarre et al. (2014)*」

正しい引用：> Vincent-Lamarre, P., Blondin Massé, A., Lopes, M., Lord, M., Marcotte, O., & Harnad, S. > (2016). The latent structure of dictionaries. > *Topics in Cognitive Science*, 8(3), 625 – 659. <https://doi.org/10.1111/tops.12211>

2014 年は arXiv プレプリント (v1) の日付です。査読済み論文は 2016 年です。引用には必ずジャーナル版の年 (2016) を使ってください。

使用辞書：Longman・Cambridge・Merriam-Webster・WordNet の 4 種。OALD・WordSmyth は本論文に登場しません。

6. Tarjan vs. Kosaraju の比較 —論文には記載なし

Tarjan と Kosaraju の比較 (Tarjan = 1 パス、Kosaraju = 2 パス + 逆グラフ) は技術的に正確ですが、この比較は原著に書かれていません。原著は Tarjan (1972) のアルゴリズムのみを名指して引用しています。

このような補足知識を加えること自体は良いことですが、発表では「論文の内容」と「自分が調べて追加した知識」を明確に分けて説明してください。

来週までのアクション

1. Blondin Massé et al. (2008) の Algorithm 2 を読み、*fast/speed/moving/quickly* の例を手でトレースして out-degree が正しいことを自分で確認してください。
2. このリポジトリの `tests/test_kernel_outdegree.py` を見てください。`test_cycle_plus_tail_buggy_w` テストが、正しいアルゴリズムとバグのあるアルゴリズムの結果の違いを示しています。
3. `src/rs_dic_llm/metrics/kernel_core.py` を見る前に、自分で `compute_kernel` を実装してみてください。